LAPORAN TUGAS DEEP LEARNING
MATA KULIAH MACHINE LEARNING
PERTEMUAN 11

Implementasi Model Neural Network Sequential Multi-Layer Perceptron untuk Klasifikasi Biner Menggunakan Dataset BlaBla



Disusun oleh:

Nabila Risha Juliana

NIM: 434241017

Kelas: TI C2

Link Github : https://github.com/nabilarishajuliana/Pertemuan11_DeepLearning

PROGRAM STUDI D4 TEKNIK INFORMATIKA

FAKULTAS VOKASI

UNIVERSITAS AIRLANGGA SURABAYA

2026

DAFTAR ISI

BAB I **3**
PENDAHULUAN **3**
1.1 Latar Belakang 3
1.2 Rumusan Masalah 3
1.3 Tujuan 3
BAB II **5**
DASAR TEORI **5**
2.1 Deep Learning dan Neural Network 5
2.2 Arsitektur Sequential Multi-Layer Perceptron 5
2.3 Fungsi Aktivasi ReLU dan Sigmoid 5
2.4 Optimizer Adam dan Binary Crossentropy 6
BAB III **7**
DESKRIPSI DATASET **7**
3.1 Sumber Dataset 7
3.2 Karakteristik dan Fitur 7
BAB IV **9**
METODOLOGI DAN IMPLEMENTASI **9**
4.1 Import Library 9
4.2 Load Dataset 9
4.3 Pisahkan Fitur dan Label 10
4.4 Encoding Umur 10
4.5 Preprocessing Data 12
4.6 Split Data 80:20 13
4.7 Bangun Model Neural Network 14
4.8 Compile dan Training Model 15
4.9 Evaluasi dan Confusion Matrix 17
BAB V **18**
HASIL DAN ANALISIS **18**
5.1 Hasil Training Model 18
5.2 Hasil Evaluasi dan Confusion Matrix 19
5.3 Analisis Classification Report 20
BAB VI **22**
KESIMPULAN **22**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam era digital saat ini, Machine Learning memainkan peran penting dalam pengambilan keputusan berdasarkan data. Salah satu tugas Machine Learning yang paling umum adalah klasifikasi biner, yaitu memprediksi apakah suatu sampel termasuk dalam kategori positif atau negatif berdasarkan sejumlah fitur input.

Deep Learning sebagai cabang dari Machine Learning yang menggunakan Neural Network dengan banyak layer, telah terbukti mampu menangkap pola-pola non-linear yang kompleks dalam data. Neural Network terinspirasi dari struktur dan cara kerja otak manusia, dimana setiap neuron menerima input, memprosesnya melalui fungsi aktivasi, dan mengirimkan output ke neuron berikutnya.

Pada tugas Pertemuan 11 ini, dibangun model Deep Learning menggunakan arsitektur Sequential Multi-Layer Perceptron (MLP) untuk melakukan klasifikasi biner pada dataset bernama BlaBla. Dataset ini terdiri dari 13 fitur input yang meliputi informasi umur (yang telah di-encoding ke dalam kategori) dan 12 fitur biner (0 atau 1). Target prediksi adalah kolom N yang berupa variabel biner (0 atau 1). Model diimplementasikan menggunakan library TensorFlow dan Keras di Python, kemudian hasilnya dievaluasi menggunakan Confusion Matrix dan Classification Report.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, dirumuskan beberapa pertanyaan penelitian sebagai berikut:

1. Bagaimana cara mengimplementasikan model Deep Learning menggunakan arsitektur Sequential Multi-Layer Perceptron untuk klasifikasi biner pada dataset BlaBla?
2. Bagaimana cara melakukan preprocessing data yang meliputi encoding fitur umur, handling missing value, dan handling outlier?
3. Bagaimana performa model Deep Learning dalam mengklasifikasikan data BlaBla berdasarkan metrik Accuracy, Precision, Recall, dan F1-Score?

1.3 Tujuan

Tujuan dari tugas ini adalah sebagai berikut:

1. Mengimplementasikan model Deep Learning dengan arsitektur Sequential MLP (13-16-8-1) untuk klasifikasi biner pada dataset BlaBla menggunakan library TensorFlow dan Keras.
2. Melakukan preprocessing data secara lengkap meliputi encoding kategori umur sesuai aturan modul, handling missing value, handling outlier menggunakan metode IQR Clipping, dan feature scaling menggunakan StandardScaler.
3. Menganalisis performa model berdasarkan nilai Accuracy, Confusion Matrix, dan Classification Report pada data testing.

BAB II

DASAR TEORI

2.1 Deep Learning dan Neural Network

Deep Learning adalah cabang dari Machine Learning yang menggunakan Artificial Neural Network (ANN) dengan banyak layer tersembunyi (hidden layers). Neural Network sendiri merupakan model komputasi yang terinspirasi dari struktur dan cara kerja neuron di otak manusia. Setiap neuron dalam jaringan menerima input, memprosesnya melalui fungsi aktivasi, dan mengirimkan output ke neuron-neuron berikutnya.

Dalam konteks klasifikasi, Neural Network dapat mempelajari pola-pola kompleks dari data melalui proses training. Selama training, model mengupdate bobot (weights) dan bias melalui algoritma backpropagation untuk meminimalkan loss function. Semakin dalam (deep) arsitektur jaringan, semakin kompleks pola yang dapat dipelajari, namun juga semakin besar risiko overfitting.

2.2 Arsitektur Sequential Multi-Layer Perceptron

Sequential Multi-Layer Perceptron (MLP) adalah arsitektur Neural Network paling dasar dimana layer-layer disusun secara berurutan dari input ke output tanpa percabangan. Setiap layer terhubung penuh (fully connected) dengan layer sebelumnya, sehingga setiap neuron menerima input dari seluruh neuron di layer sebelumnya.

Pada tugas ini, digunakan arsitektur Sequential MLP dengan tiga layer utama, yaitu dua hidden layer dengan 16 dan 8 neuron, serta satu output layer dengan 1 neuron. Jumlah neuron yang semakin kecil di layer-layer berikutnya disebut sebagai arsitektur funneling, yang berfungsi untuk mengekstraksi fitur dari dimensi tinggi ke dimensi yang lebih rendah hingga menghasilkan prediksi akhir.

2.3 Fungsi Aktivasi ReLU dan Sigmoid

Fungsi aktivasi berperan penting dalam Neural Network karena memberikan sifat non-linear pada model. Tanpa fungsi aktivasi, Neural Network sedalam apapun hanya akan setara dengan regresi linear sederhana.

Rectified Linear Unit (ReLU) adalah fungsi aktivasi yang paling umum digunakan pada hidden layer karena komputasinya sederhana dan menghindari masalah vanishing

gradient. ReLU mengoutputkan nilai input jika positif, dan nol jika negatif. Secara matematis, ReLU didefinisikan sebagai f(x) = max(0, x).

Sigmoid adalah fungsi aktivasi yang mengkompres input ke rentang 0 sampai 1, sehingga sangat cocok untuk output klasifikasi biner. Nilai output Sigmoid dapat diinterpretasikan sebagai probabilitas kelas positif. Secara matematis, Sigmoid didefinisikan sebagai f(x) = 1 / (1 + exp(-x)).

2.4 Optimizer Adam dan Binary Crossentropy

Adam (Adaptive Moment Estimation) adalah algoritma optimizer yang menggabungkan keunggulan RMSprop dan Momentum. Adam secara adaptif mengubah learning rate untuk setiap parameter berdasarkan estimasi momen pertama dan kedua dari gradien. Adam menjadi pilihan default karena cepat konvergen dan robust terhadap berbagai jenis dataset.

Binary Crossentropy adalah fungsi loss yang dirancang khusus untuk klasifikasi biner. Loss function ini mengukur jarak antara distribusi probabilitas prediksi dan distribusi probabilitas sebenarnya (label true). Semakin kecil nilai binary crossentropy, semakin baik performa model dalam memprediksi kelas yang benar. Secara matematis, Binary Crossentropy didefinisikan sebagai -(y log(p) + (1-y) log(1-p)), dimana y adalah label sebenarnya dan p adalah probabilitas prediksi.

BAB III

DESKRIPSI DATASET

3.1 Sumber Dataset

Dataset yang digunakan dalam tugas ini adalah BlaBla.xlsx yang telah disediakan oleh dosen pengampu mata kuliah Machine Learning. Dataset ini berisi data dengan 15 kolom yang meliputi informasi umur asli, kode umur yang telah di-encoding, 12 fitur biner, dan 1 kolom target yang akan diprediksi.

3.2 Karakteristik dan Fitur

Karakteristik umum dataset BlaBla.xlsx ditampilkan pada Tabel 3.1 di bawah ini.

Tabel 3.1 Karakteristik Umum Dataset BlaBla

Keterangan	Detail
Sumber	File BlaBla.xlsx dari dosen pengampu
Jumlah Baris	2.308 baris data
Jumlah Kolom	15 kolom (13 fitur input + 1 target + 1 kolom umur asli)
Tipe Data	Integer (biner 0/1 dan kategorikal untuk umur)
Target Prediksi	Kolom N (0 atau 1)
Distribusi Kelas Target	Kelas 0: 1.629 (70.58%) | Kelas 1: 679 (29.42%)
Missing Values	1 nilai non-numerik pada UMUR_TAHUN (ditangani dengan median)

Dataset BlaBla memiliki 15 kolom dengan rincian penggunaan dalam tugas ini seperti ditampilkan pada Tabel 3.2.

Tabel 3.2 Deskripsi Kolom Dataset BlaBla

No	Kolom	Keterangan Penggunaan
1	A	Kode umur hasil encoding (1-5), digunakan sebagai validasi hasil encoding
2	UMUR_TAHUN	Umur asli dalam tahun, akan di-encoding sesuai aturan modul
3	B - M	12 fitur biner (nilai 0 atau 1), digunakan sebagai input model
4	**N**	**Target output (0 atau 1), kelas yang akan diprediksi oleh model**

Sesuai dengan ketentuan modul, variabel input yang digunakan adalah Umur (hasil encoding) dan B sampai M, sehingga total terdapat 13 fitur input. Variabel output adalah kolom N.

BAB IV

METODOLOGI DAN IMPLEMENTASI

4.1 Import Library

Tahap pertama adalah mengimpor seluruh library yang dibutuhkan untuk membangun model Deep Learning, meliputi library untuk manipulasi data, visualisasi, preprocessing, dan pembangunan Neural Network.

```
import os
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
import tensorflow as tf

# Set random seed untuk reproducibility
np.random.seed(42)
tf.random.set_seed(42)
```

Library pandas dan numpy digunakan untuk manipulasi data tabular dan operasi numerik. Seaborn dan matplotlib dipakai untuk visualisasi hasil. Library sklearn menyediakan fungsi train_test_split untuk membagi data, StandardScaler untuk scaling, dan metrik evaluasi. Library TensorFlow dan Keras digunakan untuk membangun model Neural Network.

4.2 Load Dataset

Dataset BlaBla.xlsx dimuat menggunakan pandas dengan path relatif ke folder yang sama dengan file Python. Fungsi pd.read_excel digunakan karena file berformat .xlsx.

```
base_dir = os.path.dirname(os.path.abspath(__file__))
data_path = os.path.join(base_dir, "BlaBla.xlsx")
df = pd.read_excel(data_path)

print(f"Shape: {df.shape[0]} baris x {df.shape[1]} kolom")
print(f"Kolom: {list(df.columns)}")
print(df.head())
```

Hasil load dataset menunjukkan terdapat 2.308 baris data dengan 15 kolom. Screenshot output load dataset ditampilkan pada Gambar 4.1.

```
[1] Dataset dimuat
    Shape: 2308 baris x 15 kolom
    Kolom: ['A', 'UMUR_TAHUN', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N']

    Preview 5 baris pertama:
   A UMUR_TAHUN  B  C  D  E  F  G  H  I  J  K  L  M  N
0  1         17  0  1  0  0  0  0  0  1  0  0  0  1  0
1  5         70  0  0  0  0  0  0  0  1  1  1  0  1  1
2  3         39  0  0  0  0  0  1  0  0  0  0  0  1  0
3  5         63  0  0  0  0  0  0  0  0  1  0  0  1  0
4  3         40  0  0  0  0  0  1  0  0  0  1  0  1  0
```

Gambar 4.1 Output Terminal Load Dataset BlaBla

4.3 Pisahkan Fitur dan Label

Sesuai dengan ketentuan modul Pertemuan 11, variabel input adalah Umur beserta kolom B hingga M (total 13 fitur), sedangkan variabel output adalah kolom N.

```
# Variabel Input  : Umur + B sampai M = 13 fitur
# Variabel Output : N
print("Variabel Input  : Umur, B, C, D, E, F, G, H, I, J, K, L, M")
print("Variabel Output : N")
```

4.4 Encoding Umur

Kolom UMUR_TAHUN yang berupa angka tahun di-encoding menjadi kode kategori 1 sampai 5 sesuai dengan aturan pada modul Pertemuan 11, dengan aturan seperti ditampilkan pada Tabel 4.1.

Tabel 4.1 Aturan Encoding Umur

Rentang Umur	Kode Hasil Encoding
Umur kurang dari sama dengan 20	1
Umur 21 sampai 30	2
Umur 31 sampai 40	3
Umur 41 sampai 50	4
Umur lebih dari 50	5

Implementasi encoding dalam kode Python sebagai berikut:

```python
def encode_umur(umur):
    umur = int(umur)
    if umur <= 20:
        return 1
    elif umur <= 30:
        return 2
    elif umur <= 40:
        return 3
    elif umur <= 50:
        return 4
    else:
        return 5

# Pastikan UMUR_TAHUN bertipe numerik, isi NaN dengan median kalau ada
df['UMUR_TAHUN'] = pd.to_numeric(df['UMUR_TAHUN'], errors='coerce')
if df['UMUR_TAHUN'].isna().any():
    median_umur = df['UMUR_TAHUN'].median()
    df['UMUR_TAHUN'] = df['UMUR_TAHUN'].fillna(median_umur)

df['Umur_Encoded'] = df['UMUR_TAHUN'].apply(encode_umur)
```

Dalam proses encoding ditemukan 1 nilai non-numerik pada kolom UMUR_TAHUN, yaitu nilai Y pada baris 2151. Nilai tersebut dikonversi ke NaN kemudian diisi dengan median kolom UMUR_TAHUN (nilai 27) sebelum proses encoding dilakukan. Hasil encoding kemudian divalidasi dengan kolom A yang sudah tersedia di dataset, hasilnya 2307 dari 2308 baris cocok (hanya 1 baris berbeda karena penanganan nilai non-numerik). Screenshot output tahap encoding ditampilkan pada Gambar 4.2.

```
[3] Encoding UMUR_TAHUN ke Kode Umur
    Catatan: ditemukan 1 nilai non-numerik di UMUR_TAHUN, diisi median (27.0)
    Rules Encoding:
        1 = umur <= 20
        2 = umur 21-30
        3 = umur 31-40
        4 = umur 41-50
        5 = umur > 50

    Contoh hasil encoding (5 sampel):
   UMUR_TAHUN  Umur_Encoded  A
0        17.0             1  1
1        70.0             5  5
2        39.0             3  3
3        63.0             5  5
4        40.0             3  3

    Distribusi kode umur:
Umur_Encoded
1    785
2    567
3    451
4    294
5    211

    Validasi dengan kolom A: 2307/2308 baris cocok
```

Gambar 4.2 Output Terminal Proses Encoding Umur

4.5 Preprocessing Data

Preprocessing data meliputi tiga tahap utama, yaitu pengecekan missing value, handling outlier menggunakan metode IQR, dan feature scaling menggunakan StandardScaler.

4.5.1 Cek Missing Value

Dilakukan pengecekan nilai kosong (NaN) pada seluruh fitur input X dan label y. Hasil menunjukkan tidak terdapat missing value pada data input maupun label setelah tahap encoding dilakukan.

```
# a. Cek Missing Value
print(f"Total missing di X : {X.isnull().sum().sum()}")
print(f"Total missing di y : {y.isnull().sum()}")
```

4.5.2 Handling Outlier dengan IQR Clipping

Outlier dalam setiap fitur diidentifikasi menggunakan metode Interquartile Range (IQR). Nilai yang berada di luar batas Q1 - 1.5 x IQR hingga Q3 + 1.5 x IQR dianggap sebagai outlier. Metode yang digunakan adalah clipping, dimana outlier dipotong ke batas terdekat, bukan dihapus. Hal ini dilakukan untuk mempertahankan jumlah data.

```
for col in X.columns:
    Q1 = X[col].quantile(0.25)
    Q3 = X[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    X[col] = X[col].clip(lower, upper)
```

Catatan: Karena sebagian besar fitur B hingga M adalah fitur biner (0 atau 1), metode IQR akan mendeteksi nilai minoritas sebagai outlier. Namun karena nilai minoritas adalah bagian penting dari data, clipping pada fitur biner bersifat teknis dan tetap diaplikasikan sesuai prosedur modul.

4.5.3 Feature Scaling dengan StandardScaler

Tahap scaling sangat penting untuk Neural Network karena algoritma ini sensitif terhadap perbedaan skala antar fitur. StandardScaler dari sklearn digunakan untuk

mentransformasi fitur sehingga memiliki mean=0 dan standar deviasi=1. Dengan demikian, setiap fitur berkontribusi secara proporsional terhadap pembelajaran model.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Screenshot output lengkap tahap preprocessing ditampilkan pada Gambar 4.3.

```
[4] Preprocessing Data
    X (input) : (2308, 13) -> 13 fitur
    y (output): (2308,) -> target biner (0/1)

    Distribusi kelas target N:
    {0: np.int64(1629), 1: np.int64(679)}
    Rasio: {0: 70.58, 1: 29.42} %

    a. Cek Missing Value:
       Total missing di X : 0
       Total missing di y : 0
       -> Tidak ada missing value

    b. Cek Outlier (IQR):
       Umur_Encoded   : 0 outlier
       B              : 372 outlier (di-clip ke batas IQR)
       C              : 39 outlier (di-clip ke batas IQR)
       D              : 337 outlier (di-clip ke batas IQR)
       E              : 85 outlier (di-clip ke batas IQR)
       F              : 250 outlier (di-clip ke batas IQR)
       G              : 195 outlier (di-clip ke batas IQR)
       H              : 175 outlier (di-clip ke batas IQR)
       I              : 0 outlier
       J              : 297 outlier (di-clip ke batas IQR)
       K              : 0 outlier
       L              : 151 outlier (di-clip ke batas IQR)
       M              : 0 outlier

    c. Feature Scaling (StandardScaler):
       -> Semua fitur di-standardize (mean=0, std=1)
       -> Shape X_scaled: (2308, 13)
```

Gambar 4.3 Output Terminal Preprocessing Data

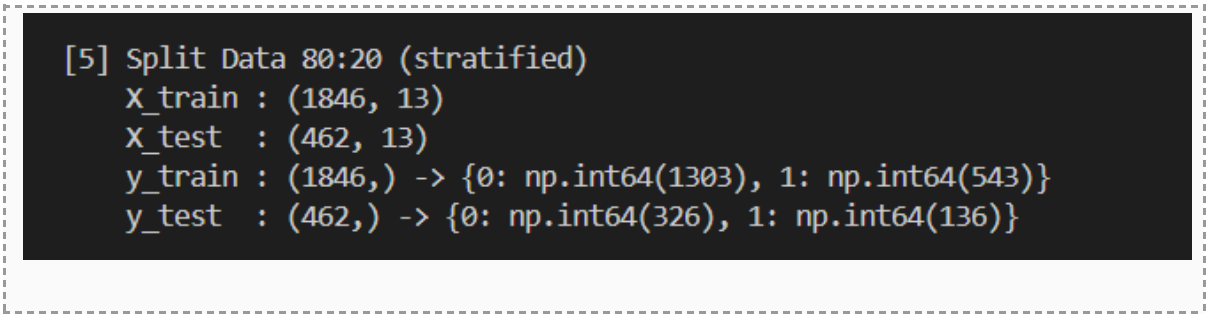
4.6 Split Data 80:20

Data dibagi menjadi 80% training dan 20% testing menggunakan fungsi train_test_split dari sklearn. Parameter stratify=y digunakan agar proporsi kelas target tetap seimbang antara data training dan testing. Parameter random_state=42 menjamin pembagian yang konsisten setiap kali kode dijalankan.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, stratify=y, random_state=42
)
```

```
print(f"X_train : {X_train.shape}")
print(f"X_test  : {X_test.shape}")
```

Hasil split data menghasilkan 1.846 sampel training dan 462 sampel testing. Screenshot output ditampilkan pada Gambar 4.4.



Gambar 4.4 Output Terminal Split Data 80:20

4.7 Bangun Model Neural Network

Arsitektur model Neural Network yang dibangun merupakan Sequential Multi-Layer Perceptron dengan tiga Dense layer. Hidden layer pertama memiliki 16 neuron dengan fungsi aktivasi ReLU, hidden layer kedua memiliki 8 neuron dengan aktivasi ReLU, dan output layer memiliki 1 neuron dengan aktivasi Sigmoid untuk klasifikasi biner.

```
model = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(16, activation='relu'),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
])

model.summary()
```

Hasil model summary menunjukkan total parameter yang dilatih adalah 369 parameter. Screenshot model summary ditampilkan pada Gambar 4.5.

```
[6] Bangun Arsitektur Neural Network
I0000 00:00:1794650319.839966   23100 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CP
U instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the
 appropriate compiler flags.

    Arsitektur Model:
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	224
dense_1 (Dense)	(None, 8)	136
dense_2 (Dense)	(None, 1)	9

```
Total params: 369 (1.44 KB)
Trainable params: 369 (1.44 KB)
Non-trainable params: 0 (0.00 B)
```

Gambar 4.5 Arsitektur Model Neural Network

4.8 Compile dan Training Model

Model dikompilasi dengan optimizer Adam, loss function binary_crossentropy yang cocok untuk klasifikasi biner, dan metric accuracy untuk monitoring performa selama training. Training dilakukan selama 100 epochs dengan validation_split=0.2, yang berarti 20% dari data training dipakai sebagai validation set untuk monitoring overfitting.

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    X_train, y_train,
    epochs=100,
    validation_split=0.2,
    verbose=1,
    batch_size=32
)
```

Selama training, nilai accuracy dan loss baik pada data training maupun validation data dicatat setiap epoch. Screenshot sebagian proses training ditampilkan pada Gambar 4.6.



Gambar 4.6 Proses Training Model Neural Network

4.9 Evaluasi dan Confusion Matrix

Setelah training selesai, model dievaluasi pada data testing menggunakan metode evaluate. Kemudian prediksi kelas dihasilkan dengan threshold 0.5, dimana jika probabilitas output lebih dari 0.5 maka diprediksi sebagai kelas 1, dan selainnya diprediksi sebagai kelas 0. Hasil prediksi kemudian dibandingkan dengan label sebenarnya untuk menghasilkan Confusion Matrix dan Classification Report.

```
# Evaluasi
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Loss     : {loss:.4f}")
print(f"Akurasi  : {acc * 100:.2f}%")

# Confusion Matrix
y_pred_proba = model.predict(X_test, verbose=0)
y_pred = (y_pred_proba > 0.5).astype(int).flatten()
cm = confusion_matrix(y_test, y_pred)

# Visualisasi Confusion Matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=[0, 1], yticklabels=[0, 1])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

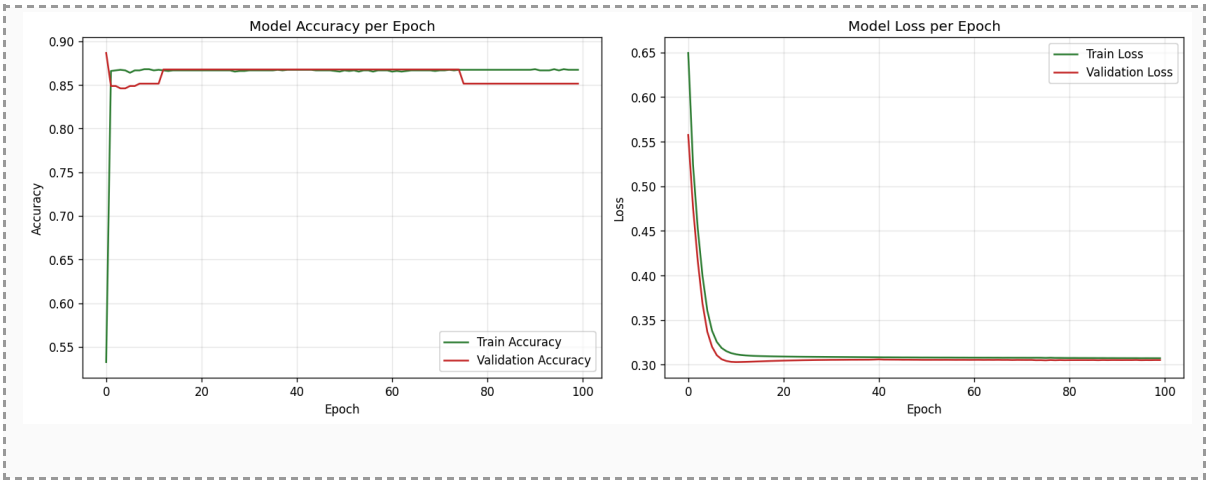
# Classification Report
print(classification_report(y_test, y_pred))
```

BAB V

HASIL DAN ANALISIS

5.1 Hasil Training Model

Proses training model dilakukan selama 100 epochs dengan validation_split=0.2. Grafik perkembangan accuracy dan loss baik pada data training maupun validation ditampilkan pada Gambar 5.1.



Gambar 5.1 Grafik Training History Model (Accuracy dan Loss per Epoch)

Berdasarkan Gambar 5.1 dapat diamati pola training sebagai berikut:

Pada grafik Model Accuracy per Epoch, training accuracy meningkat sangat cepat dari sekitar 53% pada epoch pertama menjadi sekitar 87% hanya dalam 2-3 epoch. Setelah itu, accuracy stabil pada kisaran 86-87% hingga epoch ke-100. Validation accuracy juga menunjukkan pola serupa, stabil di sekitar 85-87% setelah epoch ke-10.

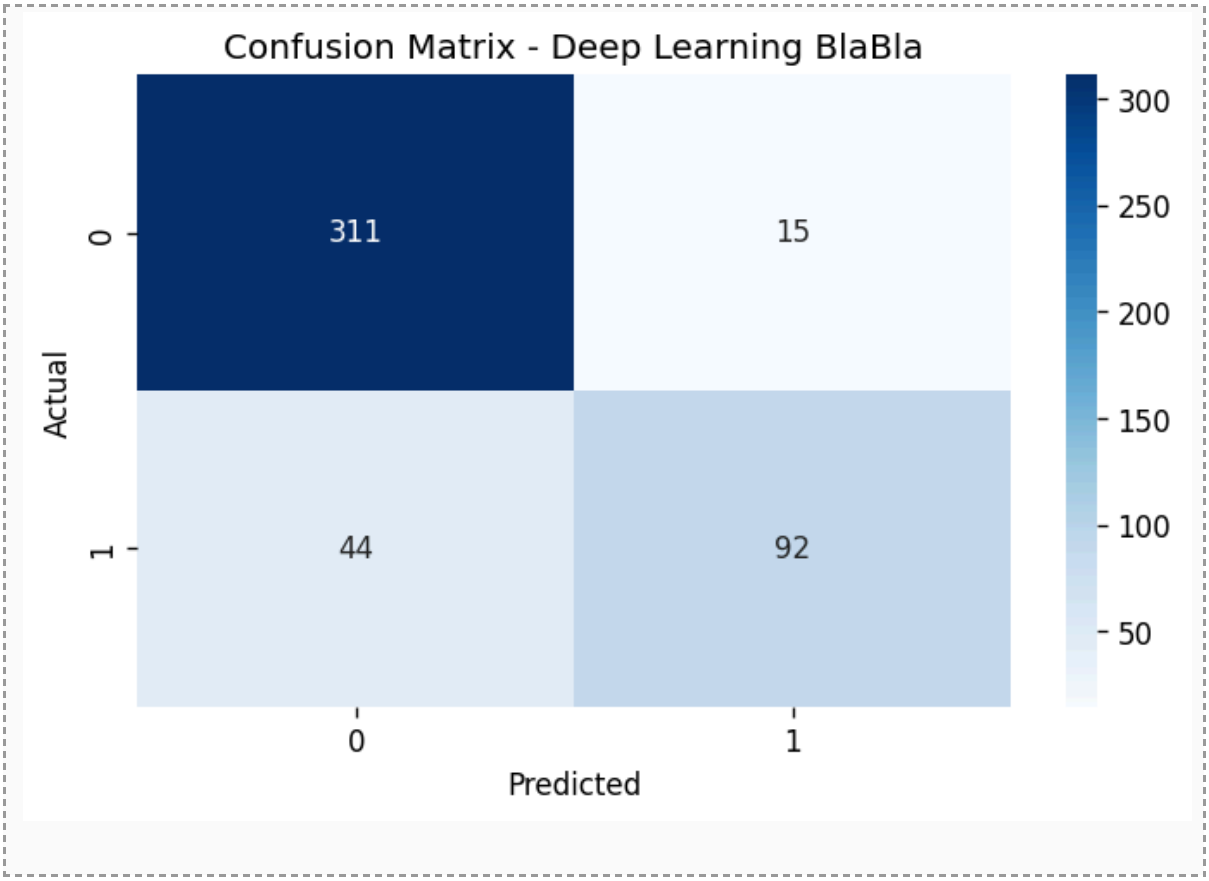
Pada grafik Model Loss per Epoch, training loss menurun sangat cepat dari 0.65 menjadi 0.31 dalam 10 epoch pertama, kemudian stabil pada nilai sekitar 0.31 hingga akhir training. Validation loss menunjukkan pola yang serupa, dan yang penting, nilai validation loss berada SANGAT DEKAT dengan training loss sepanjang training.

Pola training ini menunjukkan karakteristik yang SANGAT BAIK. Kecepatan konvergensi yang tinggi (stabil setelah 10 epoch) menunjukkan bahwa dataset memiliki pola yang jelas dan mudah dipelajari oleh model. Yang lebih penting, kedekatan antara training

dan validation metric menunjukkan bahwa model TIDAK mengalami overfitting, melainkan dapat menggeneralisasi dengan baik ke data yang belum pernah dilihat.

5.2 Hasil Evaluasi dan Confusion Matrix

Setelah 100 epochs training, model dievaluasi pada data testing yang berjumlah 462 sampel. Hasil evaluasi menunjukkan nilai loss sebesar 0.3191 dan akurasi sebesar 87.23%. Confusion Matrix dari prediksi model terhadap data testing ditampilkan pada Gambar 5.2.



Gambar 5.2 Confusion Matrix Model Deep Learning pada Data Testing

Berdasarkan Confusion Matrix pada Gambar 5.2, komponen prediksi dapat dibaca sebagai berikut yang ditampilkan pada Tabel 5.1.

Tabel 5.1 Komponen Confusion Matrix Hasil Prediksi

	Prediksi: 0	**Prediksi: 1**
Aktual: 0	TN = 311	FP = 15
Aktual: 1	FN = 44	TP = 92

Interpretasi komponen Confusion Matrix:

1. True Negative (TN=311): Model dengan benar mengklasifikasi 311 sampel sebagai kelas 0. Angka yang sangat tinggi ini menunjukkan model sangat akurat dalam mendeteksi kelas negatif.
2. False Positive (FP=15): Model keliru mengklasifikasi 15 sampel kelas 0 sebagai kelas 1. Jumlah ini sangat rendah, hanya sekitar 4.6% dari total sampel kelas 0.
3. False Negative (FN=44): Model keliru mengklasifikasi 44 sampel kelas 1 sebagai kelas 0. Angka ini merupakan jumlah kesalahan terbesar, menunjukkan model cenderung lebih berhati-hati dalam memprediksi kelas 1.
4. True Positive (TP=92): Model dengan benar mengklasifikasi 92 sampel sebagai kelas 1.

5.3 Analisis Classification Report

Classification Report memberikan analisis lebih detail berupa Precision, Recall, dan F1-Score untuk masing-masing kelas, ditampilkan pada Gambar 5.3 dan diringkas pada Tabel 5.2.

```
[12] Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.95      0.91       326
           1       0.86      0.68      0.76       136

    accuracy                           0.87       462
   macro avg       0.87      0.82      0.84       462
weighted avg       0.87      0.87      0.87       462
```

Gambar 5.3 Classification Report Model Deep Learning

Tabel 5.2 Ringkasan Metrik Evaluasi per Kelas

Kelas	Precision	Recall	F1-Score
Kelas 0	0.88	0.95	0.91
Kelas 1	0.86	0.68	0.76
Macro Average	0.87	0.82	0.84

Weighted Average	0.87	0.87	0.87
Accuracy			**0.87 (87.23%)**

Analisis Metrik per Kelas:

Kelas 0 menunjukkan performa yang sangat baik dengan Precision 0.88, Recall 0.95, dan F1-Score 0.91. Nilai Recall 0.95 berarti model berhasil mengidentifikasi 95% dari seluruh sampel yang sebenarnya berkelas 0. Ini menunjukkan model sangat jarang melewatkan kelas negatif.

Kelas 1 menunjukkan performa yang cukup baik dengan Precision 0.86, Recall 0.68, dan F1-Score 0.76. Precision 0.86 berarti ketika model memprediksi kelas 1, 86% prediksi tersebut benar. Namun Recall 0.68 menunjukkan model hanya berhasil mendeteksi 68% dari seluruh sampel yang sebenarnya berkelas 1, sisanya 32% keliru dicap sebagai kelas 0.

Perbedaan performa antar kelas ini disebabkan oleh karakteristik dataset yang imbalanced, dimana kelas 0 mendominasi dengan 70.58% sampel. Model cenderung lebih percaya diri dalam memprediksi kelas mayoritas karena melihat lebih banyak contoh selama training. Meskipun begitu, Accuracy keseluruhan sebesar 87.23% menunjukkan performa model yang sangat baik dan dapat diandalkan untuk tugas klasifikasi biner pada dataset ini.

BAB VI

KESIMPULAN

Berdasarkan hasil implementasi dan analisis model Deep Learning untuk klasifikasi biner pada dataset BlaBla, dapat disimpulkan beberapa hal sebagai berikut:

1. Model Deep Learning dengan arsitektur Sequential Multi-Layer Perceptron (13-16-8-1) berhasil diimplementasikan menggunakan library TensorFlow dan Keras. Model memiliki total 369 parameter yang dilatih melalui algoritma backpropagation dengan optimizer Adam.
2. Preprocessing data yang dilakukan meliputi encoding fitur umur ke dalam 5 kategori sesuai aturan modul (hasil encoding tervalidasi 2307/2308 cocok dengan kolom A pada dataset), handling missing value pada kolom UMUR_TAHUN dengan median, handling outlier dengan metode IQR Clipping, dan feature scaling menggunakan StandardScaler.
3. Model mencapai akurasi 87.23% pada data testing dengan nilai loss sebesar 0.3191. Untuk kelas 0 (mayoritas), model mencapai Precision 0.88, Recall 0.95, dan F1-Score 0.91. Untuk kelas 1 (minoritas), model mencapai Precision 0.86, Recall 0.68, dan F1-Score 0.76.
4. Confusion Matrix menunjukkan model berhasil mengklasifikasi 311 sampel kelas 0 dan 92 sampel kelas 1 dengan benar, dengan kesalahan prediksi 15 False Positive dan 44 False Negative.
5. Grafik training history menunjukkan model konvergen sangat cepat (stabil setelah 10 epoch) dengan nilai training dan validation metric yang sangat dekat, menandakan model TIDAK mengalami overfitting dan dapat menggeneralisasi dengan baik ke data baru.
6. Dengan Weighted Average F1-Score sebesar 0.87, model Neural Network terbukti sebagai metode yang efektif untuk tugas klasifikasi biner pada dataset ini, didukung oleh karakteristik data yang sudah terstruktur rapi dan fitur yang diskriminatif.